FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	DELA CRUZ, ROBIE D.
Section:	AN43
Professor:	MS. TAN
Date:	10/12/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.

4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
```

```
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
  - **More agents** (e.g., 10 or 20)
  - **Smaller/larger grid sizes**
  - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)

```
"""TA1: Random Walk Simulation Using AgentPy.

This version records every agent's path, supports collision avoidance,
prints final positions, and compares two scenarios.
Install dependencies with: pip install agentpy matplotlib
"""

import agentpy as ap
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

class RandomWalker(ap.Agent):
    """An agent that moves one square in a random cardinal direction."""

    def setup(self):
        size = self.model.p.grid_size
        self.position = (
            self.model.random.randrange(size),
            self.model.random.randrange(size),
        )
        # Task 1: Keep a history, starting with the initial location.
        self.path = [self.position]

class RandomWalkModel(ap.Model):
```

```
    """Random walkers on a bounded square grid."""

    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

    def step(self):
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

        # Track occupied squares when collision avoidance is enabled.
        occupied = {agent.position for agent in self.agents}

        # Move agents one at a time so each updated position affects the next.
        for agent in self.agents:
            x, y = agent.position
            possible = []
            for dx, dy in directions:
                new_x = x + dx
                new_y = y + dy

                # Keep every move inside the grid boundaries.
                if not (0 <= new_x < self.p.grid_size):
                    continue
                if not (0 <= new_y < self.p.grid_size):
                    continue

                # Task 2: In avoidance mode, do not move onto another agent.
                if self.p.avoid_collisions and (new_x, new_y) in occupied:
                    continue
                possible.append((new_x, new_y))

            if possible:
                new_position = self.random.choice(possible)
                occupied.discard(agent.position)
                agent.position = new_position
                occupied.add(new_position)

            # Task 1: Save the position after every step, including a wait.
            agent.path.append(agent.position)

def make_model(agents, grid_size, steps, avoid_collisions, seed=42):
    """Create and initialize a model for manual stepping or animation."""
    parameters = {
        "agents": agents,
        "grid_size": grid_size,
```

```
        "steps": steps,
        "avoid_collisions": avoid_collisions,
        "seed": seed,
    }
    model = RandomWalkModel(parameters)
    model.setup()
    return model

def print_results(model, title="Simulation results"):
    """Task 3: Print each agent's final position and recorded path."""
    print(f"\n{title}")
    for index, agent in enumerate(model.agents, start=1):
        print(f"Agent {index}: final position {agent.position}")
        print(f"  Path ({len(agent.path)} recorded positions): {agent.path}")

    unique_positions = len({agent.position for agent in model.agents})
    print(f"Unique occupied squares: {unique_positions} of {len(model.agents)} agents")

def animate_model(model):
    """Show the walkers and their paths as the model advances."""
    fig, ax = plt.subplots(figsize=(7, 7))
    grid_size = model.p.grid_size
    ax.set_xlim(-0.5, grid_size - 0.5)
    ax.set_ylim(-0.5, grid_size - 0.5)
    ax.set_xticks(range(grid_size))
    ax.set_yticks(range(grid_size))
    ax.set_aspect("equal")
    ax.grid(True, alpha=0.4)
    ax.set_title("Random Walk Simulation")

    colors = plt.cm.tab10.colors
    points = ax.scatter([], [], s=100)
    path_lines = [ax.plot([], [], color=colors[i % len(colors)], alpha=0.45)[0]
                  for i in range(len(model.agents))]
    points.set_color([colors[i % len(colors)] for i in range(len(model.agents))])

    def update(_frame):
        model.step()
        xs = [agent.position[0] for agent in model.agents]
        ys = [agent.position[1] for agent in model.agents]
        points.set_offsets(list(zip(xs, ys)))
        for line, agent in zip(path_lines, model.agents):
            line.set_data([p[0] for p in agent.path], [p[1] for p in agent.path])
        ax.set_xlabel(f"Step {len(model.agents[0].path) - 1}")
```

```
        return [points, *path_lines]

    # The initial positions are shown before the first movement.
    points.set_offsets([agent.position for agent in model.agents])
    for line, agent in zip(path_lines, model.agents):
        line.set_data([agent.position[0]], [agent.position[1]])

    animation = FuncAnimation(
        fig, update, frames=model.p.steps, interval=350,
        blit=False, repeat=False, cache_frame_data=False,
    )
    plt.show()
    return animation  # Keep a reference alive until the window closes.

def compare_scenarios():
    """Task 4: Compare a light-load and a crowded, longer simulation."""
    scenarios = [
        ("Scenario A: 5 agents, 10x10 grid, 20 steps", 5, 10, 20),
        ("Scenario B: 12 agents, 5x5 grid, 50 steps", 12, 5, 50),
    ]
    print("\nScenario comparison (collision avoidance enabled)")
    for name, agents, grid_size, steps in scenarios:
        model = make_model(agents, grid_size, steps, True, seed=42)
        for _ in range(steps):
            model.step()
        unique = len({agent.position for agent in model.agents})
        print(
            f"{name}: {unique}/{agents} distinct final squares; "
            f"{agents} paths recorded with {steps + 1} positions each."
        )

def main():
    print("Random Walk Simulation Using AgentPy")
    try:
        num_agents = int(input("Enter number of agents: "))
        grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
        num_steps = int(input("Enter number of steps: "))
    except ValueError:
        print("Please enter whole numbers only.")
        return

    if num_agents < 1 or grid_size < 1 or num_steps < 1:
        print("Agents, grid size, and steps must all be at least 1.")
        return
```

```
    model = make_model(num_agents, grid_size, num_steps, False)
    print("\nAnimating the main scenario. Agents may share a square in this run.")
    animate_model(model)
    print_results(model, "Main scenario: final positions and paths")
    compare_scenarios()

if __name__ == "__main__":
    main()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?

   The grid becomes crowded, so agents are more likely to occupy the same squares when collisions are allowed. If collision avoidance is turned on, agents have fewer empty neighboring squares to choose from. Some may have to stay in place when all valid neighboring squares are occupied.

2. How does increasing the number of steps affect the **spread of agents**?

   More steps give agents more chances to travel and explore different parts of the grid, so their paths usually become longer. However, random movement does not guarantee that agents will keep spreading outward. They may revisit squares or move back toward earlier locations, and the grid boundaries limit how far they can go.

3. Do the agents ever move outside the grid? Why or why not?

   No. Before moving, the program checks that the new x and y coordinates are both between 0 and grid size minus 1. A move that would cross a boundary is removed from the list of possible moves.

4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

   It can represent a simplified version of diffusion by treating agents as particles that move randomly through space. It can represent basic crowd movement by treating agents as people moving through a bounded area. The model can be extended with obstacles, preferred directions, destinations, different walking speeds, or rules for avoiding other agents. It is a simplified model and does not capture all real-world behavior.

OUTPUT:

